SPARK

SPARK: Campus Energy and Resource Management System

Project specification for Antigravity (build brief) Hackathon: Kaushal 8.0, PS-04 "Sustainable Tech Without Limits" (software category, 6-hour coding window, 4 to 6 members, live demo, no slides)

---

0. Instructions to the coding agent

You are building a working MVP, not a mockup. Read this whole document first, then:

1. Write a short implementation plan and list any assumption you make. Do not ask questions unless something blocks you.
2. Build in the order given in section 13. Finish each stage so it runs end to end before starting the next.
3. Every number on screen must come from real calculations on data in the database. No hard-coded display values and no decorative buttons.
4. Label all demo data as **SYNTHETIC** in the UI. Never present generated values as real measurements.
5. Keep the scope to this document. Items in section 12 are out of scope.
6. Write unit tests for the calculations and detection rules in sections 6 and 7. Run them and fix failures.
7. Finish with a README containing install, run and demo steps.

Reference files attached with this brief

- `spark.html`: a working front-end prototype with the screens, rules and sample data. Match its behaviour and calculations. You may improve the design.
- `occupancy_agent.py`: the OpenCV and YOLO people-counting agent. Keep its output format.
- `CRMS_Food_Waste_Management_MVP_Specification.docx`: the source specification for the food waste module.
- The problem statement image (PS-04).

---

1. Problem and goal

Problem (from PS-04): Water, electricity and materials are wasted on campuses because nobody tracks them in a way people can act on. Build a software solution that helps a campus see, understand and reduce its environmental footprint in a measurable way.

Judging expectations: a real problem clearly understood, a working core (not a static UI), visible link between the solution and its impact, deliberate technical choices, and potential to scale. A working MVP on sample or simulated data is acceptable.

SPARK's answer: one platform with three areas.

1. **Energy management:** find electricity wastage, confirm it with camera occupancy, assign actions and verify savings.
2. **Resource management:** reduce food waste in the mess or canteen.
3. **Carbon emission:** convert campus electricity use into CO_2.

Users: campus administrators, facility staff, mess manager, kitchen staff, waste logger, students (reporting only), campus reviewer.

---

2. Principles

- Measure first, then recommend. Suggestions are advisory and need human approval.
- Transparent rule-based logic. Every alert shows why it fired and a possible false positive. No machine learning in the

MVP.

- Separate **measured**, **estimated**, **simulated** and **verified** values everywhere they appear.
- Never claim a saving without a baseline and a matched comparison.
- Privacy: the camera system sends people counts only. No images, faces or identities are stored or transmitted. Reports collect no names.
- Safety: the software never controls electrical equipment automatically.
- Mobile friendly: forms must work on a phone or shared kitchen tablet.

---

3. Recommended stack

Layer	Choice	Reason
Frontend	React + Vite + Tailwind CSS + Recharts	Fast to build, good charts
Backend	Python FastAPI	Same language as the YOLO agent, automatic API docs
Database	SQLite via SQLAlchemy	Zero setup, persistent
Occupancy agent	Python, OpenCV, Ultralytics YOLO (`yolov8n.pt`)	Runs on a CPU next to the camera
Tests	pytest (backend), Vitest (optional, front end)	

If this stack is unsuitable, choose another and say why in the plan. Keep the data model and rules identical.

---

4. Information architecture

```
Main dashboard (SPARK)
 ├─ Energy management
 │    ├─ Overview
 │    ├─ Waste alerts
 │    ├─ Camera occupancy
 │    ├─ Report waste
 │    ├─ Actions
 │    └─ Savings
 ├─ Resource management (Food waste)
 │    ├─ Overview
 │    ├─ Meal planner
 │    ├─ Production
 │    ├─ Record waste
 │    ├─ Closeout
 │    ├─ Reports
 │    └─ Stock & expiry
 └─ Carbon emission
```

- The main dashboard shows the name **SPARK** and exactly three large buttons: Energy management, Resource management, Carbon emission.
- Every section has a **campus selector** (Main Campus, Engineering Campus, Hostel Campus; names editable in settings).
- Food waste screens also have a **role switcher** (demo only; see section 9).
- A persistent banner reads: "SYNTHETIC demo data" whenever seeded data is shown.

---

5. Data model

Use these tables. Add `created_at` to every table and `source` where noted.

Table	Key fields
`campus`	id, name, area_description
`zone`	id, campus_id, code (lab, lib, host, can, aud, pump), name, check_after_hours (bool), has_event_flag, equipment_note, camera_id
`energy_reading`	id, zone_id, date, total_kwh, after_hours_kwh, source (`measured` / `calculated` / `estimated` / `simulated`)

`occupancy_sample`	id, zone_id, ts, people, source (`yolo` / `simulated` / `manual`)
`waste_report`	id, zone_id, category, description, status (`unverified` / `verified`), merged_count, created_at
`alert`	id, zone_id, rule (R1 to R4), severity, avoidable_kwh, explanation, false_positive_note, status, created_at
`action`	id, alert_id, zone_id, title, owner, priority, due_date, status (`Open` / `In progress` / `Resolved`), resolution_note, resolved_at
`savings_check`	id, action_id, baseline_start, baseline_end, post_start, post_end, baseline_avg, post_avg, reduction_pct, saved_kwh, confidence, notes
`meal_session`	id, date, meal_type, location, expected_diners, actual_diners, status (`Draft` / `Open` / `Closed`), created_by, no_waste_flag
`menu_item`	id, name, std_kg_per_diner, cost_per_kg (nullable)
`session_item`	id, session_id, item_id, planned_kg, prepared_kg, served_kg
`waste_record`	id, session_id, item (or "Mixed / unclassified"), category, reason, quantity_kg, disposition, note, recorded_by, recorded_at
`suggestion`	id, item_id, meal_type, adjust_pct, decision (`accepted` / `rejected`), decided_by, decided_at
`ingredient_batch`	id, ingredient, qty_remaining_kg, received_date, best_before, storage_location
`audit_event`	id, user_role, entity, record_id, action, detail, timestamp
`settings`	tariff_inr_per_kwh, emission_factor_kg_per_kwh, thresholds

Validation rules

- Reject blank required fields, negative or non-numeric quantities, and waste weight of zero or less.
- Never overwrite a historical measurement silently. Corrections create an `audit_event` with who, when and why.
- Do not mix kg with other units without conversion.
- If actual diners is zero or missing, show waste per 100 diners as "not available".
- Duplicate meal session (same location, date and meal type) is rejected unless the role is admin.
- Show timestamps in the campus time zone (Asia/Kolkata).

---

6. Energy management: features and rules

6.1 Constants (settings, editable)

- Tariff: **₹8 per kWh (assumption).** Replace with the college's actual tariff.
- Emission factor: **0.82 kg CO_2 per kWh (assumption).** Replace with the current published CEA grid value before the final demo, and show its source in the UI.
- Baseline window: first 10 days of the 14-day series.

6.2 Overview screen

- KPIs: kWh used today, cost today, estimated avoidable kWh (sum of active alerts), active alerts and open actions.
- Table by zone: kWh today, bar, after-hours kWh, people after hours (camera), status (Alert / High but expected / Normal).
- Daily trend chart (14 days) by zone or total. Show a data-quality line (days present per zone).
- Hostel is not checked for after-hours use because residents are present by design (`check_after_hours = false`).

6.3 Detection rules (all transparent, all tested)

Rule	Logic	Output
R1 After-hours use	Let `median_ah` be the median after-hours kWh over the baseline window. Fire if `ah_today > max(2 × median_ah, median_ah + 3)`. Then read `people` = after-hours average from camera.	If `people == 0`: alert, severity High, text "Camera confirms the zone was empty". If `people` is 1 or 2: alert, severity Medium. If `people >= 3`: **no alert**; show under "expected" as rule R4 with the people count.
R2 Above baseline	Compute mean and standard deviation of daily kWh over the baseline window. Fire if `today > mean + 2.5 × sd` AND `today > 1.15 × mean`.	Alert, severity Medium. If the zone has an event flag today, do not alert; show as "High, expected".
R3	Count separate (non-merged) waste reports for the zone in	Alert, severity Medium.

Repeated reports	the last 7 days. Fire if count is 3 or more.	
R4 Occupied high use	Output of R1 when `people >= 3` (see above).	Informational only.

Each alert must show: zone, rule name, severity, plain-language reason with the actual numbers, **potential avoidable kWh, ₹ and kg CO_2 (labelled "estimate until verified")**, and a possible false-positive note. Optional equipment check example: 40 PCs × 100 W × 8 h = 32 kWh.

Formulas:

- `kWh = power_W × quantity × hours ÷ 1000`
- `avoidable_kWh = ah_today − median_ah` (R1), `today − mean` (R2)
- `cost = kWh × tariff`, `$CO_2$ kg = kWh × emission_factor`

6.4 Camera occupancy screen

- Explain the system in two sentences: a YOLO person detector runs next to each camera, counts people every minute and sends only the number.
- Table by zone: camera id, average people after hours, after-hours kWh, verdict ("Power on, nobody there" / "High use, few people" / "High use, people present: expected" / "Normal" / "Not evaluated").
- Hourly chart for a chosen zone: electricity as bars, people as a line, red bars where power is on and nobody is counted overnight.
- **Load camera counts:** a paste box and API endpoint that accept the agent's JSON lines. Lines with timestamps between 22:00 and 05:00 set that zone's after-hours people value; alerts recompute immediately. Reject lines with an unknown zone, negative people or an invalid timestamp, and report how many lines were accepted.

6.5 Waste reporting (QR)

- Each zone has a QR code that opens the report form for that zone. Provide a page that lists QR codes for printing.
- Form: zone, issue (lights left on, fans running in empty room, computers left on, AC running unnecessarily, outdoor lights on in daylight, other), optional details (max 200 characters). No name is collected.
- Merge same-zone, same-issue reports on the same day into one report with a count.
- Staff can verify a report. Show the notice: "Do not touch electrical equipment. Only report what you see."
- Rate-limit submissions per device to limit spam.

6.6 Actions

- Create an action from an alert with default owner, priority (High for R1, Medium otherwise) and due date (3 days).
- Statuses: Open, In progress, Resolved. Resolving requires a resolution note.
- After resolving, prompt for follow-up readings. "Resolved" never counts as "saved".
- Optional: escalate an action that stays Open past its due date.

6.7 Savings verification

- For a resolved action, the user picks a baseline period (issue active) and a post-fix period.
- Compute `baseline_avg` and `post_avg` of after-hours kWh per night, then:
  - `reduction % = (baseline_avg − post_avg) ÷ baseline_avg × 100`
  - `saved kWh = (baseline_avg − post_avg) × number of post days`
  - cost and CO_2 saved using the settings
- Show confidence (Low, Medium, High) and notes. Medium means meter-based but occupancy, timetable and weekday mix are not controlled. Require at least 3 post days before showing a result; otherwise show "estimate only, collect more readings".
- Seed one fully worked example (Library after-hours fix) so the screen is never empty.
- Show a monthly projection only as a labelled projection, never as verified.

---

7. Resource management: Food waste (from the CRMS specification)

7.1 Roles and permissions

Role	Can	Cannot
CRMS admin	Manage users and settings, view all reports, allow duplicate sessions	Change waste records without an audit trail
Mess manager	Create plans, close meals, correct records (with reason), accept or reject suggestions	Delete historical records
Kitchen staff	Enter planned and prepared quantities, manage stock	Change settings
Waste logger	Record waste entries	See personal student data
Campus reviewer	View dashboards and export	Edit anything

For the MVP, implement a role switcher that enforces these permissions in the API, not just in the UI.

7.2 Screens

1. **Overview:** period filter (last 7 days / all); KPI cards (total waste kg, waste per 100 diners, diners served, estimated cost, kg donated); waste by category; top 5 wasted items; trend of waste per 100 diners by day; plan vs actual (expected vs actual diners, planned vs prepared kg); data-quality prompt listing sessions missing diners, production or closeout.
2. **Meal planner:** date, meal, expected diners (prefilled with the average of the last four comparable meals), standard menu with kg per diner. Planned quantity = expected diners × standard kg per diner. Save as Draft. Advisory suggestions panel (7.4).
3. **Production:** planned, prepared and served kg per item. If prepared is more than 125% of plan, show a warning but never block. Changing a saved value requires a reason and creates an audit event.
4. **Record waste:** item (or "Mixed / unclassified"), category, kg (large number input), reason, disposition, optional note. Prevent accidental double submission.
5. **Closeout:** actual diners, production totals, waste entry count, missing-field check. Separate **Save draft** and **Close meal**. Closing requires actual diners, prepared quantity for every item, and either at least one waste entry or a "No waste to record" tick. Show the dining-hall camera count (aggregate YOLO estimate, may include staff) with a "Use camera count" button.
6. **Reports:** filters by meal type, category and item; CSV export; baseline vs pilot comparison with a note that no reduction is claimed without matched periods; audit trail.
7. **Stock & expiry:** batches with expiry status (OK, expires within 2 days, expired); add batch; "Log as spoiled" on an expired batch creates a waste record in the category "Spoiled / expired stock".

7.3 Categories and rules

- Waste categories: Preparation scraps; Unserved cooked surplus; Plate returns; Spoiled / expired stock; Other.
- Reasons: Overproduction; Low demand; Quality / taste; Excessive portion; Spoilage; Handling issue; Unknown.
- Dispositions: General disposal; Compost / biowaste; Approved donation; Other approved route; Unknown.
- **Food with disposition "Approved donation" is never counted as waste.** Show it separately as "donated".

7.4 Calculations

- Total waste kg = sum of waste records, excluding approved donations.
- Waste per 100 diners = waste kg ÷ actual diners × 100, using only sessions with actual diners above zero.
- Unserved-surplus rate (%) = unserved cooked surplus kg ÷ prepared cooked food kg × 100.
- Plate waste per diner = plate-return kg ÷ actual diners.
- Estimated cost = sum of kg × cost per kg. Records without a cost are excluded and counted separately, never treated as zero.
- **Suggestion rule:** for a meal type, look at the last four closed comparable meals. If an item's unserved surplus exceeds 10% of its prepared quantity in at least 3 of them, suggest reducing its standard quantity by 5% (10% if the average rate exceeds 20%). Suggestions are advisory and need manager acceptance. Never cut production automatically. Record every accept or reject in the audit trail.
- Expected diners fallback: average actual diners of the last four comparable sessions; if none exist, manual entry.

---

8. Carbon emission screen

- Monthly electricity (kWh) = sum over zones of average daily kWh × 30, labelled estimate.

- Emissions = kWh × emission factor, shown in tonnes CO_2.
- Show the factor and its source. Show trees-equivalent as a rough conversion (about 21 kg CO_2 per tree per year), labelled rough.
- Optional: add the food-waste footprint later. Do not invent a food emission factor without a cited source.

---

9. API surface (suggested)

```
GET  /api/campuses
GET  /api/energy/overview?campus=
GET  /api/energy/alerts?campus=
POST /api/occupancy                 (JSON lines or array from the agent)
GET  /api/occupancy/zone/{id}?date=
POST /api/reports                   (waste report)
PATCH /api/reports/{id}/verify
POST /api/actions                   (from alert)
PATCH /api/actions/{id}             (status, note)
POST /api/savings                   (baseline and post periods)
GET  /api/carbon?campus=
GET/POST /api/food/sessions         PATCH /api/food/sessions/{id}
PUT  /api/food/sessions/{id}/production
POST /api/food/waste                PATCH /api/food/waste/{id}   (needs reason)
POST /api/food/sessions/{id}/close
GET  /api/food/overview?range=
GET  /api/food/suggestions?meal=    POST /api/food/suggestions/{id}/decision
GET  /api/food/reports.csv?filters
GET/POST /api/food/stock            POST /api/food/stock/{id}/discard
GET  /api/audit
POST /api/seed                      (reload demo data)
```

All write endpoints validate input and return clear error messages. Role is sent in a header for the demo and checked against the permission table.

---

10. Camera occupancy integration

- The agent `occupancy_agent.py` runs on a small computer next to each CCTV camera: `python occupancy_agent.py --source <rtsp url> --zone lab`.
- It writes one JSON line per sample: `{"zone":"lab","ts":"2026-10-12T23:00:05","people":0,"source":"yolo"}`.
- Provide two ways to ingest: (a) paste into the UI, (b) a small script or flag that POSTs the lines to `/api/occupancy`.
- Occupancy is used for: confirming or suppressing R1, the Camera occupancy screen, and the dining-hall diner estimate.
- Cameras can miss people who are hidden, so every camera-based statement says "verify before acting".
- Do not store or stream video. Only counts leave the camera computer.

---

11. Seed (demo) data

Generate deterministic, clearly labelled SYNTHETIC data for each campus, with a campus multiplier (Main 1.0, Engineering 1.35, Hostel 0.85).

Energy (14 days, 6 zones): Computer Lab 2 (30 kWh/day), Library (45), Hostel A (90), Canteen (35), Auditorium (20), Water pumps (25), with weekend factors. Scenarios that must appear:

- **Computer Lab 2:** after-hours spike on the last 3 nights with 0 people, so R1 fires as High. Two seeded waste reports so one more report triggers R3.
- **Water pumps:** gradual rise of about 35% over the last 4 days, so R2 fires (possible fault).
- **Auditorium:** about 2.2× use today with an event flag, so R2 does not alert and shows as expected.
- **Library:** an earlier after-hours problem (days 3 to 6) resolved on day 7 with an action and note, so the Savings screen shows a worked verified example. Today it also has a high after-hours reading with about 34 people (exam night), so it shows as expected with no alert.
- **Hostel A:** not evaluated for after-hours use.

Food (about 7 days, lunch and dinner): diners about 86% to 96% of expected, rice prepared about 8% to 16% above need so it repeatedly shows unserved surplus (triggers the suggestion), plate returns on dal and roti, preparation scraps on sabzi, some rice with disposition "Approved donation" (must not count as waste), one open Draft lunch for today, and four stock batches including one expired and one expiring soon.

---

12. Out of scope for this MVP

Automatic equipment control, smart-meter or IoT integration, machine learning forecasting, face recognition, automatic donation matching, procurement or accounting replacement, photo uploads, occupancy-based weather adjustment, multi-institution tenancy.

---

13. Build order (6-hour plan)

Time	Task
0:00 to 0:30	Plan, project scaffold, database models, seed script
0:30 to 1:30	Backend: energy data, detection rules R1 to R4 with unit tests, occupancy ingest
1:30 to 3:00	Frontend: dashboard, Energy Overview, Alerts, Camera occupancy, Report waste, Actions, Savings
3:00 to 4:30	Food waste: sessions, production, waste, closeout, overview, suggestions, stock, roles, audit
4:30 to 5:15	Carbon screen, CSV export, QR page, validation and empty states, mobile check
5:15 to 6:00	Integration test of the demo script, README, polish

Fallback if the backend fails: run the same logic client-side with browser storage, keeping the identical screens and calculations.

---

14. Tests to include

- R1 fires for after-hours 31 kWh against median 3 (people 0), is a Medium alert for 1 or 2 people, and produces no alert for 3 or more people.
- R2 fires for a value above mean + 2.5 sd and 15% above mean, and is suppressed with an event flag.
- R3 fires at 3 reports in 7 days and not at 2; same-day duplicates merge.
- Savings: baseline 18 and post 4 kWh per night over 7 days gives 77.8% reduction and 98 kWh saved.
- Equipment estimate: 40 × 100 W × 8 h = 32 kWh.
- Food: waste per 100 diners with 9 kg waste and 300 diners is 3.0 kg; approved donation is excluded from waste; missing diners returns "not available".
- Suggestion appears only with 3 of 4 comparable meals above 10% surplus.
- Closing a meal fails without actual diners or prepared quantities.
- A role without permission receives a 403 from the API.
- Occupancy ingest rejects bad lines and accepts valid ones.

---

15. Demo script (3 to 5 minutes, no slides)

1. Open SPARK and show the three buttons.
2. Energy management, Overview: point to Computer Lab 2 using power overnight.
3. Camera occupancy: show power on and zero people, and the Library exam night correctly shown as expected.
4. Waste alerts: open the lab alert, show the evidence and the ₹ and CO_2 estimate.
5. Create an action, add a resolution note and resolve it.
6. Savings: show the Library fix with a baseline vs after comparison, and explain "estimate" vs "verified".
7. Resource management: record a rice leftover, close the meal, show waste per 100 diners and the advisory suggestion.
8. Carbon emission: show the monthly CO_2 and how savings reduce it.
9. Close with scaling: real meters and cameras replace the simulated feed.

16. Definition of done

- All three areas run end to end with persisted data and the seed scenarios above.
- All detection rules and calculations pass their tests.
- Role permissions are enforced by the API.
- Camera counts can be loaded and change the alerts.
- Every estimated, simulated and verified value is labelled.
- The app works on a phone-sized screen.
- README explains setup, running, seeding and the demo.

17. Assumptions to replace before the final demo

- Tariff of ₹8 per kWh and food cost of ₹90 per kg (placeholder; per-item costs in the seed).
- Emission factor of 0.82 kg CO_2 per kWh: replace with the current published CEA value and cite it in the UI.
- Thresholds in section 6.3 and 7.4 are starting values; tune them against real data.
- All consumption, occupancy and food data are synthetic until real meter readings, bills and camera counts are loaded.